Software Architecture and Design Specification (SAD): TaskBoard

Oct 28, 2026 · @harsh

TaskBoard is a three-layer web application. A vanilla-JavaScript single-page UI calls a Flask REST API, which stores data in SQLite through SQLAlchemy. It is built in two one-week sprints and released as a Docker image.

Field	Value
Project	TaskBoard, a mini project management tool
Version	1.0
Authors	[Member 1], [Member 2], Harsh Agarwal (Member 3), [Member 4]
Date	28-10-2026
Status	Draft for Sprint 1

Revision history

Version	Date	Author	Change summary
1.0	28-10-2026	Team	First SAD issued before Sprint 1 development

Approvals

Role	Name	Signature / Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of TaskBoard v1.0 and shows how each requirement in SRS v1.0 is realised. Developers build from it, and the tester derives integration tests from its API contract and sequence diagrams. Evaluators use it to check that the implementation follows the agreed design.

1.2 Scope

It covers the whole product: the browser UI, the REST API, the data model, persistence, security controls, and the CI/CD pipeline. Features listed as out of scope in SRS 1.2 are excluded: notifications, attachments, drag and drop, real-time sync, fine-grained roles, and external integrations.

1.3 Audience

Developers (Members 1 and 2), the tester (Member 3), the repo manager (Member 4), the course instructor, and assessment evaluators.

1.4 Definitions

Term	Meaning
SAD, SRS, STP, RTM	Software Architecture and Design specification, Software Requirements Specification, Software Test Plan, Requirements Traceability Matrix
ADR	Architecture Decision Record: a decision, the alternatives rejected, and why
API, REST, JSON	Application Programming Interface, Representational State Transfer, JavaScript Object Notation
SPA	Single-page application: one HTML page whose content is redrawn by JavaScript
ORM	Object-Relational Mapper (SQLAlchemy)

Term	Meaning
Blueprint	A Flask group of routes registered under one URL prefix (here /api)
App factory	The create_app() function that builds and configures the Flask app
CI, CD	Continuous Integration, Continuous Delivery (GitHub Actions)
GHCR	GitHub Container Registry, where the release Docker image is published
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
XSS, CSRF	Cross-Site Scripting, Cross-Site Request Forgery
Issue, Sprint, Backlog, Story points, Project key	As defined in SRS 1.4

2. Document overview

2.1 How to use this document

Section 3 describes the architecture: what the parts are, why they were chosen, and how they meet the requirements. Section 4 describes the design: how the parts interact, the full API contract, error handling, and the UI.

Requirement IDs follow the SRS (TB-F-###, TB-NF-###, TB-SR-###). The "Section ref / Design spec" column of the SRS RTM points at section numbers in this document. Section 3.8 gives the reverse mapping, plus the FR-## IDs used in the team playbook and STP.

2.2 Related documents

Document	Version	Relation
TaskBoard SRS	1.0, 27-10-2026	Requirements this design realises

Document	Version	Relation
TaskBoard STP (Software Test Plan)	1.0, 28-10-2026	Test cases that verify this design
RTM	SRS section 8	Requirement-to-design-to-test mapping
Team Playbook	Current	Code, Git workflow, sprint plan
README.md	In repository	Setup, run, and test instructions

3. Architecture

3.1 Goals and constraints

The architecture favours a setup every member can run in minutes over scalability the project does not need.

Goal	How the architecture meets it	Requirement
Runs on every laptop	One Python process, a file database, no external services at run time	TB-NF-003
Fast enough	Each list request is a single filtered SQL query returning compact JSON; 1,000 issues per project is small for SQLite	TB-NF-001 (95% under 500 ms)
Reliable	Data lives in a SQLite file that survives restarts; input is validated before any write; bad requests get 4xx	TB-NF-005
Secure by default	Hashed passwords, a session check on every protected route, ORM bound parameters, text-only rendering	TB-SR-001 to 007
Testable and maintainable	Every endpoint is reachable through Flask's test client on an in-memory database; CI runs on every PR	TB-NF-004

Constraints:

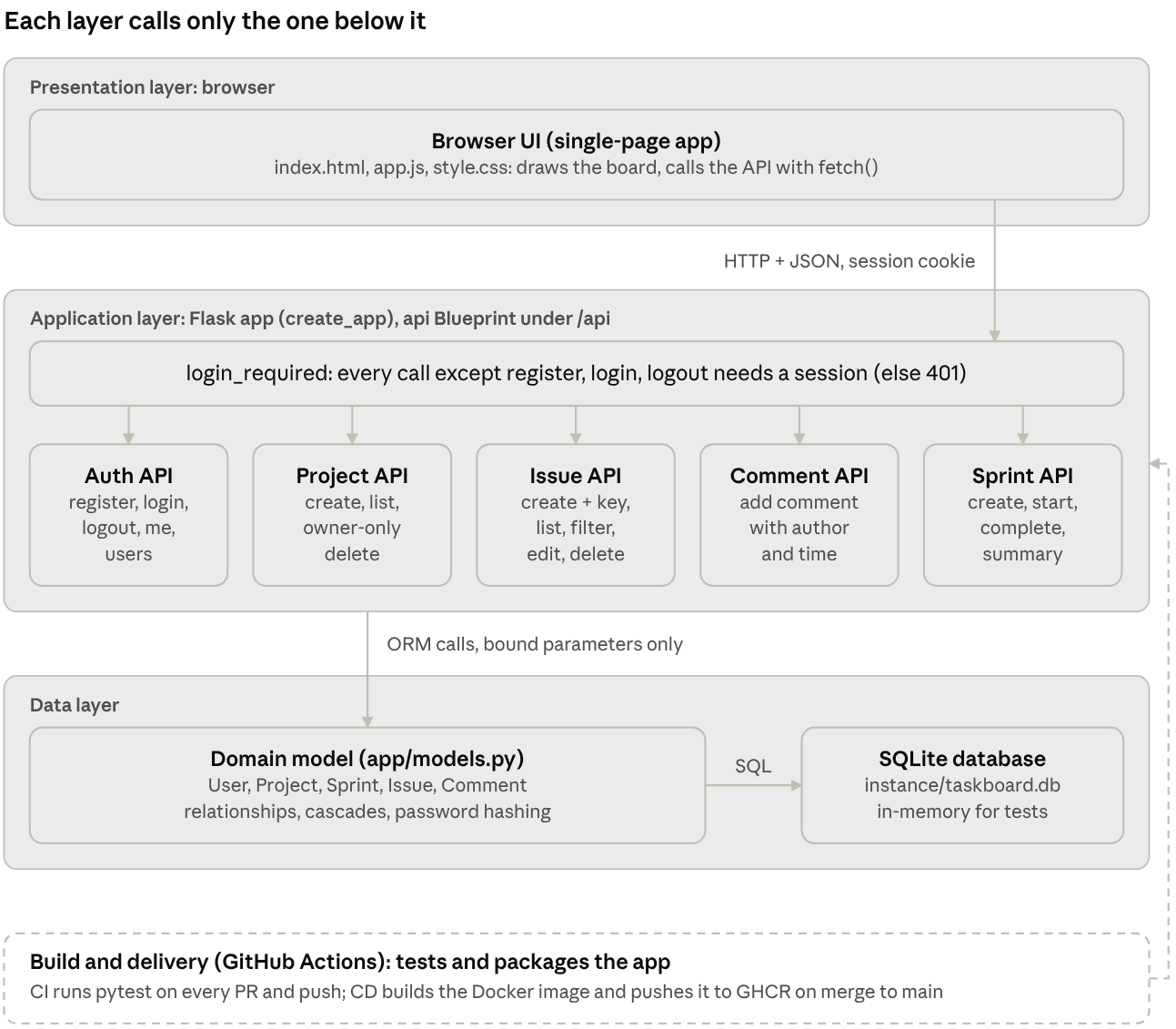
- Two one-week sprints: 27 to 31 Oct and 2 to 7 Nov 2026
- Stack fixed to Flask, SQLAlchemy, SQLite, and vanilla JavaScript
- Single server instance; a signed session cookie identifies the user
- Source on GitHub; CI and CD on GitHub Actions
- Users inside a project trust each other, so any logged-in user can view and edit any issue

If TC-Perf-01 misses its target, the first remedy is an index on issue.project_id and issue.sprint_id, recorded in docs/deviations.md.

3.2 Stakeholders and concerns

Stakeholder	Main concerns
Team members (end users)	Quick issue creation, a clear board, correct issue keys
Project owner	Nobody else can delete their project
Tester (Member 3)	A testable API, predictable behaviour, an isolated test database
Developers (Members 1, 2)	Clear module boundaries and a stable API contract
Repo manager (Member 4)	Automated CI/CD, a reproducible build, a runnable Docker image
Course instructor and evaluators	Traceability to the SRS, adherence to the design, documented deviations

3.3 Component (UML) diagram



TaskBoard components · 3 layers, 5 API components, build pipeline

The browser reaches the server only over HTTP and JSON, every API call passes the session guard first, and only the ORM touches SQLite.

3.4 Component descriptions

All five API components live in one Flask Blueprint named api in app/routes.py; they are separate logical components, not separate files.

Component	Files	Responsibility	Depends on
Browser UI	static/index.html, app.js, style.css	Login panel, project selector, sprint toolbar, create-issue bar, Kanban board; calls the API with fetch(); inserts all user text through the el() helper (textContent)	REST API over HTTP/JSON
App factory	app/**init**.py (create_app)	Configures Flask, database URI, and secret key; registers the api Blueprint under /api; serves index.html; creates tables at start-up; accepts a test config	All server components
Auth API	/register, /login, /logout, /me, /users; login_required decorator	Creates accounts, verifies passwords, sets and clears session user_id, rejects unauthenticated calls with 401	User model, session cookie
Project API	/projects, /projects/{id}	Create and list projects; owner-only delete with cascade	Project model
Issue API	/projects/{id}/issues, /issues/{id}; validate_issue	Create with generated key, list with filters, get with comments, partial update, delete	Issue, Project, User, Sprint models
Comment API	/issues/{id}/comments	Adds a comment with author and time	Comment model

Component	Files	Responsibility	Depends on
Sprint API	/projects/{id}/sprints, /sprints/{id}/start, /complete, /summary	Create, start (one active per project), complete (unfinished issues to backlog), points summary	Sprint, Issue models
Domain model (ORM)	app/models.py	Entities, relationships, cascades, allowed values (STATUSES, TYPES, PRIORITIES, POINTS), password hashing, to_dict() serialisers	SQLite through SQLAlchemy
Database	instance/taskboard.db	Persistent storage; tests use sqlite:///:memory:	None
Test suite	tests/test_api.py	API tests through the Flask test client	create_app
CI/CD pipeline	.github/workflows/ci.yml, cd.yml, Dockerfile	pytest on every push and PR to develop and main; builds and pushes the Docker image to GHCR on push to main	GitHub, GHCR

3.5 Chosen architecture pattern and rationale

TaskBoard uses a layered (three-tier) client-server architecture with a REST API. The presentation layer is the browser SPA, the application layer is the Flask Blueprint (routing, authentication, validation, business rules), and the data layer is the SQLAlchemy models over SQLite. Each layer calls only the one below it: the UI never touches the database, routes never build SQL, and models hold no HTTP logic.

ADR	Decision	Rejected	Reason
ADR-1	Layered monolith	Microservices	Four people, two weeks, one deployable; extra services add network calls, deployment and test cost with no benefit at this scale
ADR-2	JSON REST API with a vanilla JS SPA	Server-rendered Jinja templates; React	The API can be tested with pytest alone (TB-NF-004); no build step or Node toolchain, so every member can run it
ADR-3	SQLite through SQLAlchemy	PostgreSQL, MySQL	Zero install, one file per machine (TB-NF-003); the ORM gives bound parameters (TB-SR-006) and a path to swap databases later
ADR-4	Signed Flask session cookie	JWT tokens in JavaScript	Built into Flask, no token handling in client code, and a single server needs no shared session store
ADR-5	Issue keys from a per-project counter (issue_counter)	Counting existing issues	Keys are never reused after a delete, so TB-1 always means the same issue
ADR-6	Docker image published to GHCR as CD	Cloud hosting (Render, Heroku)	No extra accounts or cost; anyone runs the release with one docker run command

Maintainability (TB-NF-004) follows from this split: one module per layer, PEP 8 style, at least one test per endpoint in tests/test_api.py, and CI as a required check before merge.

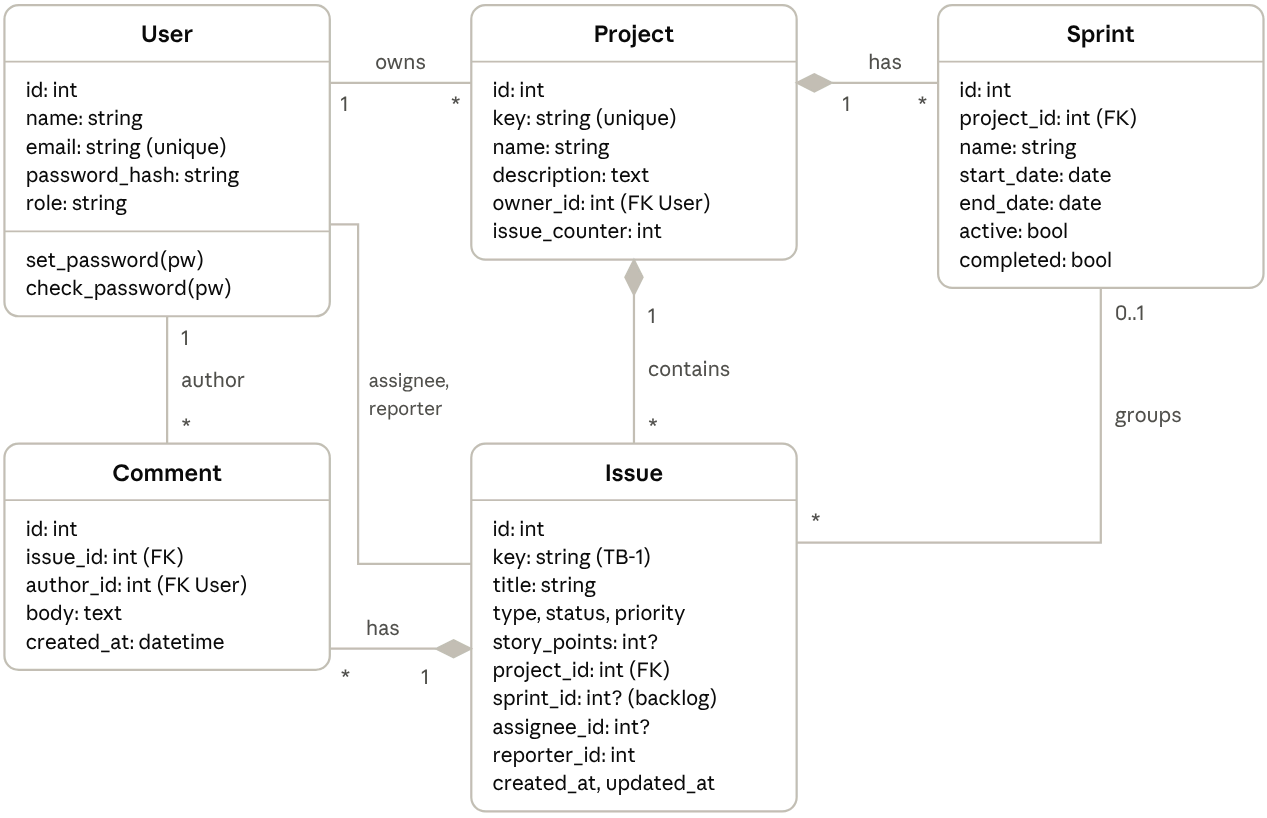
3.6 Technology stack and data stores

Layer	Technology	Version
Language	Python	3.10 or newer (CI and Docker use 3.11)
Web framework	Flask	3.0.3
ORM	Flask-SQLAlchemy (SQLAlchemy)	3.1.1
Password hashing	Werkzeug security (generate_password_hash, check_password_hash)	Bundled with Flask
Database	SQLite: instance/taskboard.db; sqlite:///:memory: for tests	Bundled with Python
Frontend	HTML5, CSS3, vanilla JavaScript (fetch API), no framework or build step	Current browsers
Tests	pytest	8.3.2
Release server	gunicorn on python:3.11-slim	22.0.0
CI/CD	GitHub Actions, GitHub Container Registry	Hosted
Process	Jira Scrum board, GitHub Project backlog	Hosted

Portability (TB-NF-003) comes from pure-Python dependencies pinned in requirements.txt, no OS-specific calls, and the Docker image as a fallback on any machine.

The data store holds five entities. The class diagram below shows them; the table lists the rules each one enforces.

Issue is the hub: every other entity links to it



UML class diagram · 5 entities, 7 associations

Deleting a project removes its issues and sprints, and deleting an issue removes its comments; an issue with no sprint is in the backlog.

Entity	Key fields	Rules
User	id, name, email, password_hash, role	Email unique and stored in lowercase; only the hash is stored, never the password
Project	id, key, name, description, owner_id, issue_counter, created_at	Key unique, 2 to 10 letters, stored uppercase; delete cascades to its issues and sprints
Sprint	id, project_id, name, start_date, end_date, active, completed	end_date not before start_date; at most one active sprint per project (checked by the Sprint API)

Entity	Key fields	Rules
Issue	id, key, project_id, sprint_id, title, description, type, status, priority, story_points, assignee_id, reporter_id, created_at, updated_at	sprint_id null means backlog; type in {Epic, Story, Task, Bug}; status in {To Do, In Progress, In Review, Done}; priority in {Low, Medium, High}; points in {1, 2, 3, 5, 8, 13} or empty; delete cascades to comments
Comment	id, issue_id, author_id, body, created_at	Body must not be empty

3.7 Risks and mitigations

Risk	Mitigation
Two short sprints leave little slack	Sprint 1 delivers the full core; Sprint 2 adds only sprints; buffer day 1 Nov; feature freeze 5 Nov
Frontend and backend drift apart	Section 4.3 is the contract; API tests in CI fail on a breaking change
SQLite allows one writer at a time	One server, short transactions committed per request; demo traffic is a few users
Secret key is hardcoded in create_app	Read it from an environment variable in Sprint 2 (TB-SR-008); logged in docs/deviations.md until then
A wrong-typed input crashes a route with 500	Type checks in validation and tests with wrong-type payloads (STP 5.1)
Laptops differ in setup	Pinned versions, the Docker image, and a final regression from a clean clone
Test runs touch real data	Tests use an in-memory database; instance/ and *.db are git-ignored

3.8 Traceability to requirements

SRS requirement	Playbook / STP ID	Realised by	SAD section
TB-F-001 to 004 (register, login, logout)	FR-01, FR-02	Auth API, User model	3.4, 4.3.2
TB-F-005 to 007 (projects)	FR-03	Project API, Project model	4.3.3
TB-F-008, 009 (create and validate issue)	FR-04	Issue API, validate_issue	4.2 Sequence 1, 4.3.4
TB-F-010 (edit, change status)	FR-06, FR-07	Issue API PATCH, card status dropdown	4.1, 4.3.4
TB-F-011 (delete issue)	FR-07	Issue API DELETE	4.3.4
TB-F-012 (assign)	FR-08	Issue API, User model	4.3.4
TB-F-013 (list and filter)	FR-13	Issue API list filters	4.3.4
TB-F-014 (Kanban board)	FR-05	Browser UI board	4.1
TB-F-015 (comments)	FR-09	Comment API, Comment model	4.3.4
TB-F-016 (create sprint)	FR-10	Sprint API, Sprint model	4.3.5
TB-F-017 (start sprint)	FR-11	Sprint API	4.3.5
TB-F-018 (complete sprint)	FR-11	Sprint API	4.2 Sequence 2, 4.3.5
TB-F-019 (sprint summary)	FR-12	Sprint API summary, UI toolbar	4.3.5
TB-F-020 (sprint filter on board)	FR-10, FR-13	UI toolbar, Issue API filter	4.1
TB-NF-001 (performance)	NFR-02	API, SQLite	3.1
TB-NF-002 (usability)	NFR-01	Browser UI	4.5
TB-NF-003 (portability)	NFR-05	Whole app, Dockerfile	3.6
TB-NF-004 (maintainability)	NFR-04	Tests, CI	3.5

SRS requirement	Playbook / STP ID	Realised by	SAD section
TB-NF-005 (reliability)	None	API, SQLite	4.4
TB-NF-006 (browsers)	None	Browser UI	4.5
TB-SR-001 to 008 (security)	NFR-03	See STRIDE table	3.9

3.9 Security architecture

Security rests on four controls: hashed credentials, a session guard on every protected route, server-side validation through the ORM, and text-only rendering. The only trust boundary is between the browser and the API, so every request body, query string, and cookie is treated as untrusted.

STRIDE threat	Example in TaskBoard	Control	Requirement
Spoofing	Logging in as another user	Salted Werkzeug password hashes; signed, HttpOnly session cookie; one generic login error	TB-SR-001, TB-SR-007
Tampering (data)	Forged cookie, SQL injection, invalid field values	Cookie signed with SECRET_KEY; ORM bound parameters only; allowed-value checks in validate_issue and each route	TB-SR-003, TB-SR-006
Tampering (script injection)	An issue titled with a script tag	The UI builds every element with textContent through el(), never innerHTML	TB-SR-004
Repudiation	Denying who created an issue or comment	reporter_id on issues; author_id and created_at on comments; no full audit log (accepted, out of scope)	None

STRIDE threat	Example in TaskBoard	Control	Requirement
Information disclosure	Leaking password hashes or which emails exist	to_dict() never returns password_hash; the same message for a wrong email and a wrong password; HTTPS beyond localhost	TB-SR-001, 007, 008
Denial of service	A malformed payload crashes the server	All input validated and answered with 4xx; no rate limiting (accepted for localhost scope)	TB-SR-003, TB-NF-005
Elevation of privilege	A logged-out user calls the API; a non-owner deletes a project	login_required returns 401; the owner check returns 403	TB-SR-002, TB-SR-005

CSRF is a low residual risk. Write endpoints read only application/json bodies, which a cross-site form cannot send, and DELETE or PATCH from another site needs a CORS preflight the server never approves.

Two gaps remain open:

- The secret key is hardcoded until the Sprint 2 change for TB-SR-008.
- A non-string title or a non-object JSON body raises an unhandled error (500), which breaks SO-3. The fix is tracked in 4.6.

4. Design

4.1 Design overview

Every user action follows one path. A UI event calls fetch() on /api; the route checks the session, validates input, and reads or writes through the ORM. It then returns JSON, and the UI redraws the board.

Module structure:

```
taskboard/
  app/
    __init__.py      app factory: create_app()
    models.py        User, Project, Sprint, Issue, Comment
    routes.py        api Blueprint: auth, projects, issues, comments, sprints
  static/
    index.html       single page
    app.js           state, api() wrapper, rendering
    style.css        layout and card styles
  tests/test_api.py  pytest API tests
  .github/workflows/ ci.yml, cd.yml
  Dockerfile, requirements.txt, run.py, README.md
```

Frontend design:

- One state object holds user, users, projects, sprints, and projectId.
- The api() wrapper sends JSON and throws an Error carrying the server's error message on any non-2xx response.
- loadBoard() fetches the current project's issues, applying the sprint filter. It groups them into four columns by status, each header showing a count (TB-F-014).
- After any change (add, status change, delete) the UI calls the API and then reloads the board. It never patches the page in place, so the board always matches the database.
- The sprint filter offers All issues, Backlog (sprint_id=none), and each sprint (TB-F-020). With a sprint selected, the toolbar shows its done and total points.

Backend design:

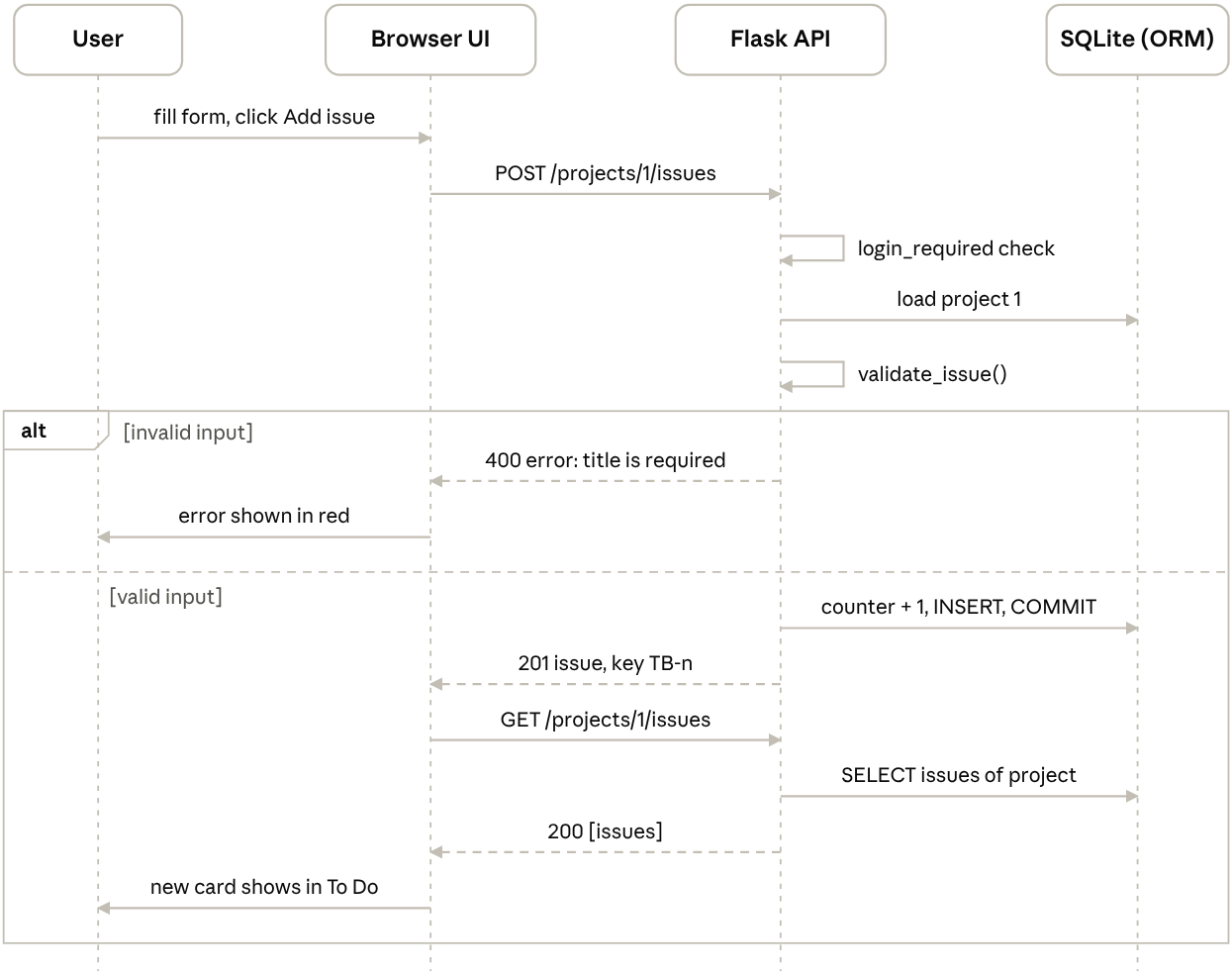
- Each protected route runs the same pipeline: login_required, then body() (silent JSON parse, empty object by default), then validation, then the change, one commit, to_dict(), and jsonify.
- Business rules sit in the routes:
  - Issue key generation increments project.issue_counter in the same transaction as the insert.
  - Only one sprint per project can be active.
  - Completing a sprint returns non-Done issues to the backlog.
  - Only the owner can delete a project.

Status workflow: To Do, then In Progress, then In Review, then Done; a reopened bug returns to To Do. The API accepts a change between any two of the four statuses, so the order is a team convention, not an enforced rule.

4.2 UML sequence diagrams

Sequence 1 shows creating an issue (TB-F-008, TB-F-009). Sequence 2 shows completing a sprint (TB-F-018), the flow the SRS RTM references as SAD 4.2 Sequence 2.

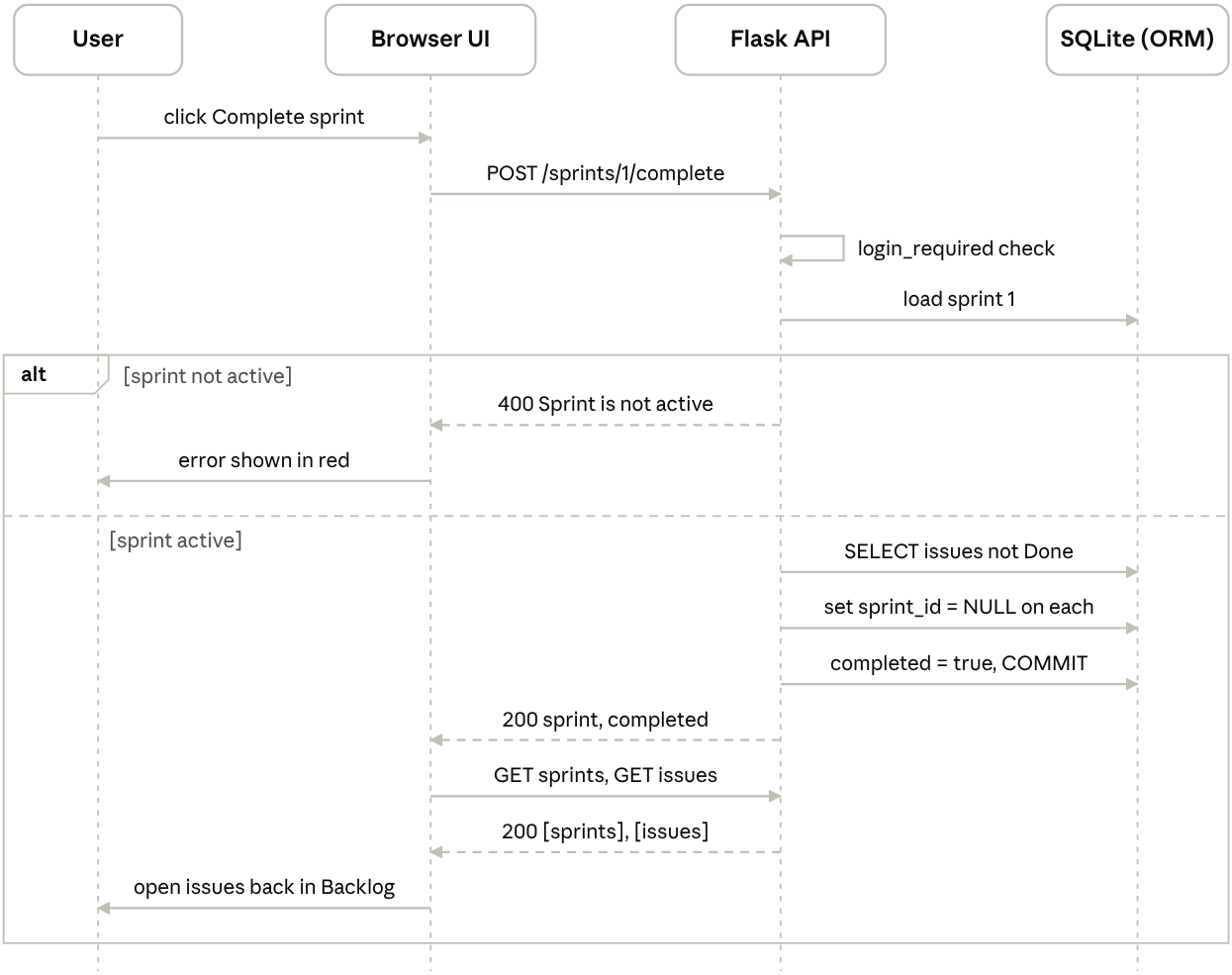
A rejected issue writes nothing; a valid one gets the next key



Sequence 1: create an issue (TB-F-008, TB-F-009)

Validation runs before any write, so a rejected request changes nothing. The counter increment and the insert share one commit, so two issues never get the same key.

Completing a sprint sends every unfinished issue back to the backlog



Sequence 2: complete a sprint (TB-F-018)

Done issues keep their sprint_id, so the sprint keeps a record of finished work. Every other issue in the sprint is moved to the backlog in the same commit.

4.3 API design

4.3.1 Conventions

- Base path /api; request and response bodies are JSON (Content-Type: application/json).
- Register and login set a signed session cookie. Every endpoint except register, login, and logout returns 401 {"error": "Login required"} without it (TB-SR-002). Logout is open because it only clears the session.

- Errors always have the shape {"error": "<message>"}. Several validation messages are joined with "; ".
- An unknown id or route returns 404 {"error": "Not found"}.
- Success codes: 201 for a create, 200 otherwise; deletes return {"ok": true}.

Method	Endpoint	Purpose	Success	Errors
POST	/api/register	Create account and log in	201	400, 409
POST	/api/login	Log in	200	401
POST	/api/logout	Log out	200	None
GET	/api/me	Current user	200	401
GET	/api/users	All users (assignee list)	200	401
GET	/api/projects	List projects	200	401
POST	/api/projects	Create project	201	400, 401, 409
DELETE	/api/projects/{id}	Delete project (owner only)	200	401, 403, 404
GET	/api/projects/{id}/issues	List issues, with filters	200	401, 404
POST	/api/projects/{id}/issues	Create issue	201	400, 401, 404
GET	/api/issues/{id}	Issue with comments	200	401, 404
PATCH	/api/issues/{id}	Update issue fields	200	400, 401, 404
DELETE	/api/issues/{id}	Delete issue	200	401, 404
POST	/api/issues/{id}/comments	Add comment	201	400, 401, 404
GET	/api/projects/{id}/sprints	List sprints	200	401, 404
POST	/api/projects/{id}/sprints	Create sprint	201	400, 401, 404
POST	/api/sprints/{id}/start	Start sprint	200	400, 401, 404, 409
POST	/api/sprints/{id}/complete	Complete sprint	200	400, 401, 404

Method	Endpoint	Purpose	Success	Errors
GET	/api/sprints/{id}/summary	Points summary	200	401, 404

4.3.2 Auth API

Endpoint	Request	Response	Errors
POST /api/register	{"name": "Asha", "email": "asha@x.com", "password": "secret1"}	201 {"id": 1, "name": "Asha", "email": "asha@x.com", "role": "Developer"} and session started	400 missing name, email without @, or password under 6 characters; 409 email already registered (case ignored)
POST /api/login	{"email": "asha@x.com", "password": "secret1"}	200 user object and session started	401 "Invalid email or password", the same for both cases (TB-SR-007)
POST /api/logout	None	200 {"ok": true}	None
GET /api/me	None	200 user object	401
GET /api/users	None	200 list of user objects, sorted by name	401

4.3.3 Project API

Endpoint	Request	Response	Errors
GET /api/projects	None	200 [{"id": 1, "key": "TB", "name": "TaskBoard", "description": "", "owner_id": 1}]	401
POST /api/projects	{"key": "tb", "name": "TaskBoard", "description": "optional"}	201 project object; key stored as "TB"; caller becomes owner	400 key not 2 to 10 letters or name missing; 409 key already exists

Endpoint	Request	Response	Errors
DELETE /api/projects/{id}	None	200 {"ok": true}; its issues, comments, and sprints are removed	403 caller is not the owner (TB-SR-005); 404

4.3.4 Issue and Comment API

Endpoint	Request	Response	Errors
GET /api/projects/{id}/issues	Optional query: status, type, assignee_id, sprint_id; sprint_id=none or assignee_id=none selects empty values	200 list of issue objects in creation order	404 project not found
POST /api/projects/{id}/issues	{"title": "Login page", "type": "Story", "priority": "High", "story_points": 3, "assignee_id": 2, "sprint_id": null}; only title is required	201 issue object with generated key, for example "TB-1"	400 validation messages (below)
GET /api/issues/{id}	None	200 issue object plus a comments list	404
PATCH /api/issues/{id}	Any of: title, description, type, status, priority, story_points, assignee_id, sprint_id; other fields are ignored	200 updated issue object	400 validation messages
DELETE /api/issues/{id}	None	200 {"ok": true}; its comments are removed	404

Endpoint	Request	Response	Errors
POST /api/issues/{id}/comments	{"body": "Looks good"}	201 {"id", "issue_id", "author", "body", "created_at"}	400 "Comment cannot be empty"; 404

An issue object looks like this:

```
{
  "id": 1, "key": "TB-1", "project_id": 1, "sprint_id": null,
  "title": "Login page", "description": "",
  "type": "Story", "status": "To Do", "priority": "High", "story_points": 3,
  "assignee_id": 2, "assignee": "Ravi", "reporter_id": 1,
  "created_at": "2026-10-28T09:30:00", "updated_at": "2026-10-28T09:30:00"
}
```

Field	Rule	Default	Error message
title	Required, not blank	None	title is required
type	Epic, Story, Task, or Bug	Task	invalid type
status	To Do, In Progress, In Review, or Done	To Do	invalid status
priority	Low, Medium, or High	Medium	invalid priority
story_points	1, 2, 3, 5, 8, 13, or null	null	story_points must be one of 1, 2, 3, 5, 8, 13
assignee_id	An existing user, or null	null	assignee not found
sprint_id	A sprint in the same project, or null	null	sprint not found in this project

4.3.5 Sprint API

Endpoint	Request	Response	Errors
GET /api/projects/{id}/sprints	None	200 list of sprint objects	404

Endpoint	Request	Response	Errors
POST /api/projects/{id}/sprints	{"name": "Sprint 1", "start_date": "2026-10-27", "end_date": "2026-10-31"}	201 {"id", "project_id", "name", "start_date", "end_date", "active": false, "completed": false}	400 dates not in YYYY-MM-DD form, name missing, or end before start
POST /api/sprints/{id}/start	None	200 sprint with active true	400 sprint already completed; 409 another sprint already active
POST /api/sprints/{id}/complete	None	200 sprint with active false and completed true; non-Done issues get sprint_id null	400 sprint not active
GET /api/sprints/{id}/summary	None	200 {"sprint": {...}, "total_points": 8, "done_points": 3, "by_status": {"To Do": 1, "In Progress": 0, "In Review": 0, "Done": 1}}	404

A summary taken after completion counts only the Done issues that stayed in the sprint, so the team records the summary before clicking Complete.

4.4 Error handling, logging and monitoring

Invalid input is rejected before anything is written, so a failed request leaves the data unchanged (TB-SR-003).

Status	Meaning	Returned when
400	Invalid input	Validation fails: missing title, bad type or date, empty comment, sprint not active

Status	Meaning	Returned when
401	Not authenticated	No session, or wrong login details
403	Not allowed	A non-owner deletes a project
404	Not found	Unknown id or route (app-wide JSON handler)
409	Conflict	Duplicate email or project key; a second active sprint
500	Unexpected error	Target is zero; any 500 found in testing is logged as a bug

- **Transactions.** Each request commits once, at the end, so an error before the commit changes nothing.
- **UI errors.** api() throws the server's message, which the UI shows in red in #authError on the login panel or #appError above the board. The next successful action clears it.
- **Reliability (TB-NF-005).** Data lives in instance/taskboard.db. db.create_all() at start-up creates missing tables without touching existing rows, and deleting the instance folder resets the data.
- **Logging.** The development server and gunicorn each print one access line per request (method, path, status). Passwords, hashes, and session cookies are never logged. Debug mode runs only through run.py on a laptop, never in the Docker image.
- **Monitoring.** There is no production deployment, so monitoring covers the pipeline and the process:
  - CI status on every PR, as a required check
  - the CD run and GHCR package page after each merge to main
  - the TC-Perf-01 timing script for response time
  - the Jira burndown for progress

4.5 UX design

The whole product is one screen: after login, everything a user needs sits on the board page.

```
+-------------------------------------------------------------------------+
| TaskBoard  [TB - TaskBoard v] [New project]          Asha  [Logout]   |
+-------------------------------------------------------------------------+
| [All issues v] [New sprint] [Start sprint] [Complete sprint]  3 / 8 points
done |
| [Issue title.............] [Story v] [Medium v] [Points v] [Assignee v]
[Add issue] |
| (error message, red)                                                    |
+-----------------+-----------------+-----------------+------------------+
| TO DO (2)       | IN PROGRESS (1) | IN REVIEW (0)   | DONE (1)         |
| |TB-1 Story High|                 |                 |                  |
| |Login page     |                 |                 |                  |
| |Ravi | 3 pts   |                 |                 |                  |
| |[To Do v]      |                 |                 |                  |
| |[Delete]       |                 |                 |                  |
+-----------------+-----------------+-----------------+------------------+
```

- Login and register share one panel; only register needs the name field.
- Each card shows key, type, and priority on the first line, the title in bold, then assignee and points. It has a status dropdown and a Delete button that asks for confirmation.
- A coloured stripe on the left of each card marks its type: blue for Task, green for Story, red for Bug, purple for Epic.
- New project and New sprint use browser prompts instead of extra screens. A new sprint defaults to today through today plus 7 days.
- Below 800 px wide the board collapses to one column (SRS 3.1).
- Usability (TB-NF-002): register, create a project, and add an issue are three actions on one screen, targeted under 2 minutes for a new user.
- Compatibility (TB-NF-006): only standard DOM and fetch APIs are used, so the page runs unchanged in current Chrome, Firefox, and Edge.
- Native inputs and buttons are keyboard-operable. Form fields use placeholders instead of label elements, a known accessibility gap (4.6).
- Out of scope, as the SRS allows: drag and drop; a comment panel (comments are API-only); editing fields other than status in the UI.

4.6 Open issues and next steps

Open issue	Impact	Plan	Sprint
Secret key hardcoded in create_app	TB-SR-008 not met; recorded deviation	Read SECRET_KEY from an environment variable; keep the default only for local development	2
A non-string title or a non-object JSON body (for example [1]) raises a 500	Breaks TB-NF-005 and SO-3	Type checks in body() and validate_issue, plus tests with wrong-type payloads	1 (bug tickets)
Starting an already-active sprint returns 409 "Another sprint is already active"	Misleading message	Return 400 "Sprint is already active"	2
No index on issue.project_id and issue.sprint_id	Possible TB-NF-001 risk at 1,000 issues	Add indexes if TC-Perf-01 fails	2
Session cookie SameSite not set	Hardening gap	Set SESSION_COOKIE_SAMESITE to Lax with the TB-SR-008 change	2
No label elements on form fields	Weaker screen-reader support	Add labels if time allows	2

Any change from this design goes in docs/deviations.md and is explained in the final demo.

Future work, beyond v1.0: drag and drop on the board, notifications, roles and permissions, a hosted deployment with HTTPS, and PostgreSQL for multi-user load.

5. Appendices

5.1 Glossary

See section 1.4 and SRS 1.4: SAD, SRS, STP, RTM, ADR, API, REST, JSON, SPA, ORM, Blueprint, CI/CD, GHCR, STRIDE, XSS, CSRF, Issue, Sprint, Backlog, Kanban board, Story points, Project key.

5.2 References

- ISO/IEC/IEEE 42010, Systems and software engineering: Architecture description
- IEEE 1016, Software Design Descriptions
- TaskBoard SRS v1.0 and STP v1.0
- TaskBoard Team Playbook
- OWASP Top 10
- Flask documentation
- SQLAlchemy documentation

5.3 Tools

- Diagrams: drawn in this document (UML component, class, and sequence); PlantUML or draw.io for re-exports
- API checks: pytest, Postman or curl
- Build and delivery: GitHub Actions, Docker, GitHub Container Registry
- Planning and tracking: Jira, GitHub Projects